



佛山大学
FOSHAN UNIVERSITY



计算机与人工智能学院
School of Computer Science and Artificial Intelligence

Section 5

NLP—句法分析

Natural Language Processing — Introduction

任课教师：陈荟慧

目录 CONTENTS



01

句法分析
概述

02

上下文无关
文法CFG

03

CFG的应用



引子

- **两句话的BOW是一样的，但意思完全不同**
 - Query 1: 谢霆锋的儿子是谁?
 - Query 2: 谢霆锋是谁的儿子?
- 因此，需要进行句法分析。
- 句法分析的主要任务：
 - 识别出句子所包含的句法成分以及这些成分之间的关系，一般以句法树来表示句法分析的结果。



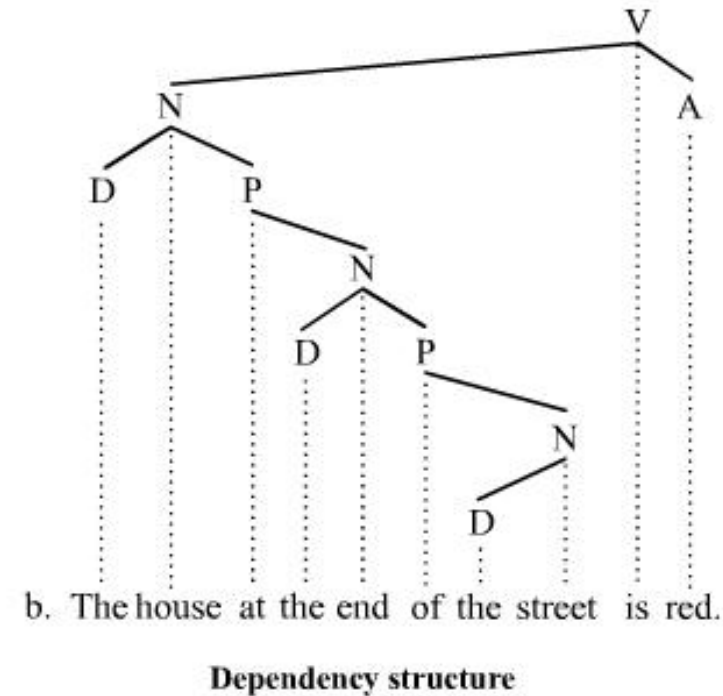
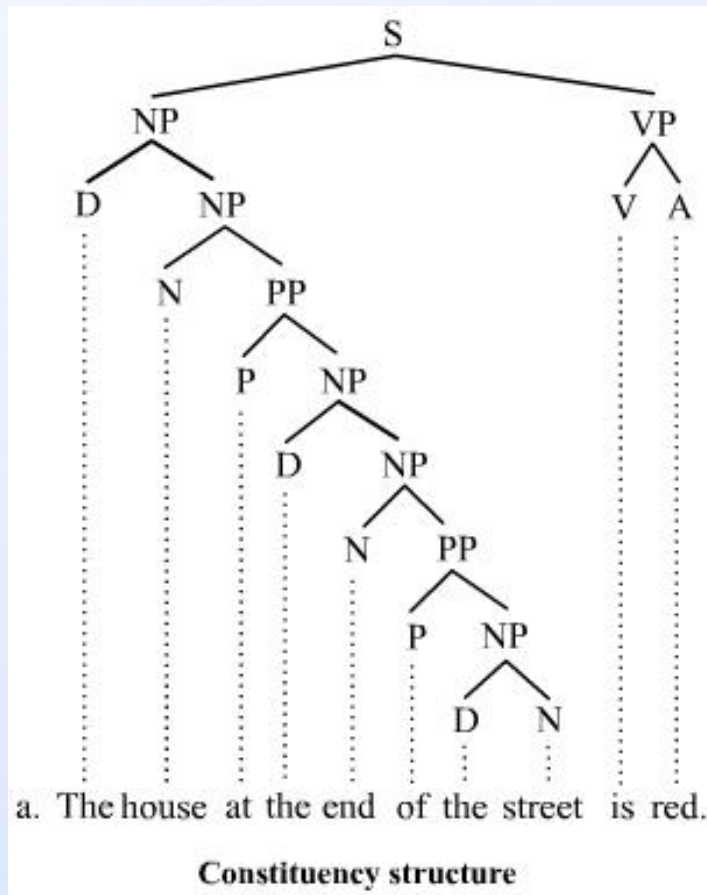


句法分析 (Parse)

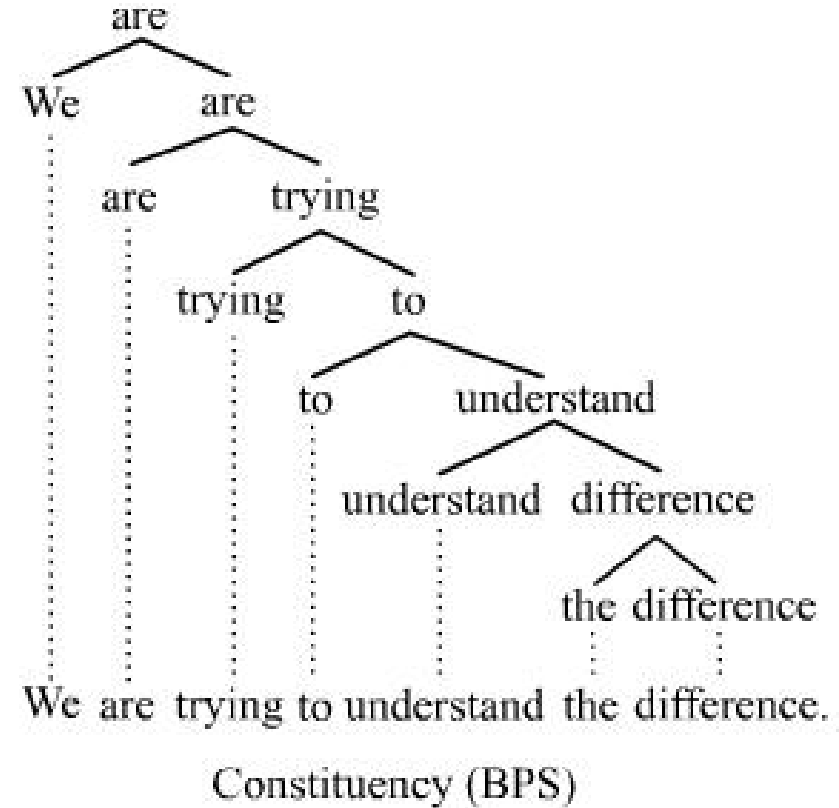
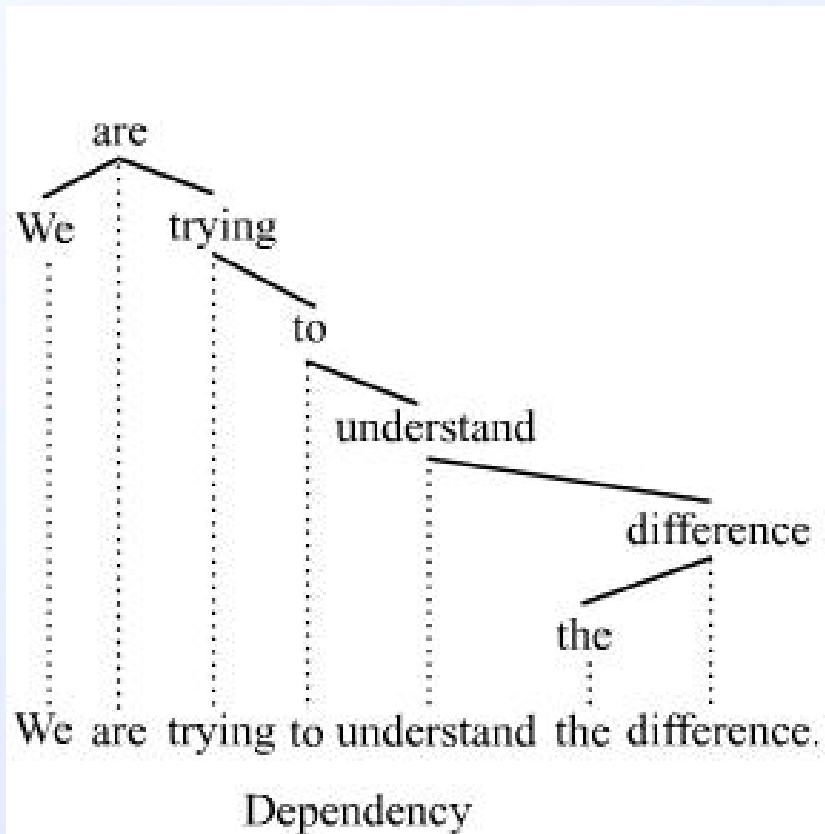
- 句法分析 (Parsing) 是从单词串得到句法结构的过程，而实现该过程的工具或程序被称为句法分析器 (Parser) 。
- 句法分析分为句法结构分析 (syntactic structure parsing) 和依存关系分析 (dependency parsing)。
 - 以获取整个句子的句法结构或者完全短语结构为目的的句法分析，被称为成分结构分析 (constituent structure parsing) 或者短语结构分析 (phrase structure parsing) ；
 - 是以获取局部成分为目的的句法分析，被称为依存分析 (dependency parsing) 。
 - 参考： https://blog.csdn.net/qq_28031525/article/details/79187080



- Most syntactic treebanks annotate variants of either phrase structure (left) or dependency structure (right).



Dependency tree and Constituency tree



$$A = \pi r^2$$



宾州树库 (Penn Treebank)

- 示例，详见：宾州中文树库标记以及其解释.pdf
 - <https://blog.csdn.net/neutblue/article/details/7375085>

标记	英语解释	中文解释
AD	adverbs	副词
AS	Aspect marker	体态词，体标记（例如：了，在，着，过）
BA	把 in ba-const	“把”，“将”的词性标记
CC	Coordinating conjunction	并列连词，“和”
CD	Cardinal numbers	数字，“一百”
CS	Subordinating conj	从属连词（例子：若，如果，如...）
DEC	的 for relative-clause etc	“的”词性标记



表 1 汉语成分标记集

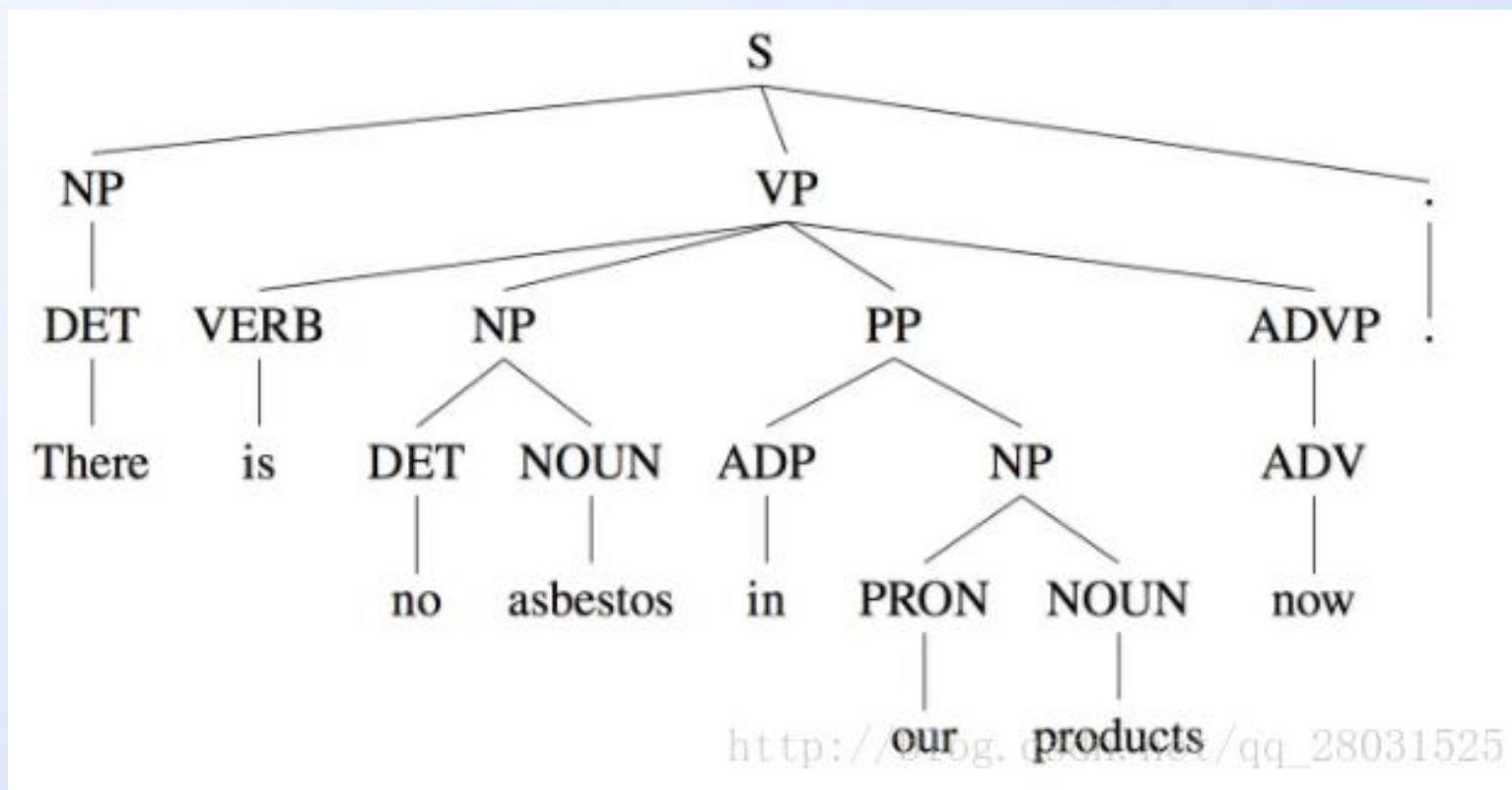
序号	标记代码	标记名称	序号	标记代码	标记名称
1	np	名词短语	9	mbar	数词准短语
2	tp	时间短语	10	mp	数量短语
3	sp	空间短语	11	dj	单句句型
4	vp	动词短语	12	fj	复句句型
5	ap	形容词短语	13	zj	整句
6	bp	区别词短语	14	jq	句群
7	dp	副词短语	15	dlc	独立成分
8	pp	介词短语	16	yj	直接引语

表 2 句法关系标记集

序号	标记代码	标记名称	序号	标记代码	标记名称
1	ZW	主谓结构	15	SX	顺序结构
2	PO	述宾结构	16	BL	并列关系
3	SB	述补结构	17	LG	连贯关系
4	DZ	定中结构	18	DJ	递进关系
5	ZZ	状中结构	19	XZ	选择关系
6	LH	联合结构	20	YG	因果关系
7	LW	连谓结构	21	MD	目的关系
8	AD	附加结构	22	JS	假设关系
9	CD	重叠结构	23	TJ	条件关系
10	JY	兼语结构	24	ZE	转折关系
11	JB	介宾结构	25	JZ	解注关系
12	FW	方位结构	26	LS	流水关系
13	KS	框式结构	27	XX	缺省关系
14	BH	标号结构			

Parsing数据集

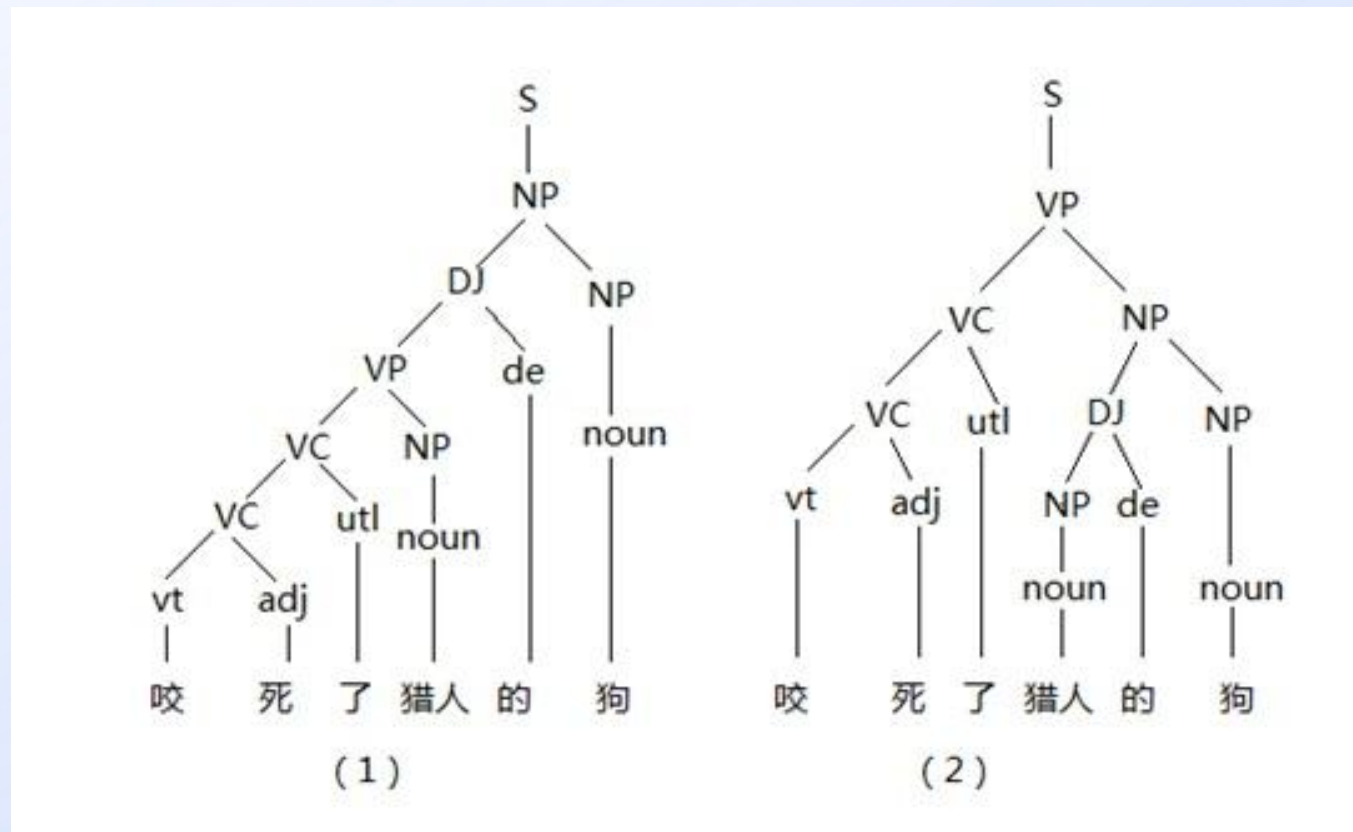
- 树库 (Treebank)
 - 句法分析的数据集是一种树形的标注结构。



$$A = \pi r^2$$

句法二义性的消除

- 例如：咬死了猎人的狗



是否可以计算哪种分析树的概率更高？

$$\frac{A}{r} = \pi r^2$$



基于PCFG的句法分析

- CFG (Context-free grammar)
 - 上下文无关文法 (CFG)，在计算机科学中，若一个形式文法 $G = (N, \Sigma, P, S)$ 的产生式规则都取如下的形式： $V \rightarrow w$ ，其中 $V \in N$ ， $w \in (N \cup \Sigma)^*$ 。
 - 上下文无关文法取名为“上下文无关”的原因就是因为字符 V 总可以被字符串 w 自由替换，而无需考虑字符 V 出现的上下文。
 - 一个形式语言是上下文无关的，如果它是由上下文无关文法生成的。





上下文无关文法的形式化定义

形式化的上下文无关文法定义：

一个四元数组 $G=(V_N, V_T, S, P)$

V_N : 非空有限的非终结符集合

V_T : 非空有限的终结符集

S : 开始符号

P : 产生式集合

其中, $V_N \cap V_T = \emptyset$, $S \in V_N$

P 中产生式一般形式为 $A \rightarrow \alpha | \beta$, 其中 $A \in V_N$, $\alpha, \beta \in (V_N \cup V_T)^*$





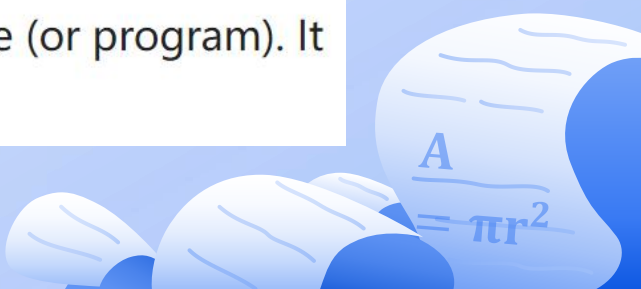
CFG

Formal definitions [[edit](#)]

A context-free grammar G is defined by the 4-tuple:^[5]

$G = (V, \Sigma, R, S)$ where

1. V is a finite set; each element $v \in V$ is called a *nonterminal character* or a *variable*. Each variable represents a different type of phrase or clause in the sentence. Variables are also sometimes called syntactic categories. Each variable defines a sub-language of the language defined by G .
2. Σ is a finite set of *terminals*, disjoint from V , which make up the actual content of the sentence. The set of terminals is the alphabet of the language defined by the grammar G .
3. R is a finite relation from V to $(V \cup \Sigma)^*$, where the asterisk represents the [Kleene star](#) operation. The members of R are called the (*rewrite*) *rules* or *productions* of the grammar. (also commonly symbolized by a P)
4. S is the start variable (or start symbol), used to represent the whole sentence (or program). It must be an element of V .


$$A = \pi r^2$$



上下文无关文法

- 一个上下文无关语法如何定义一个语言呢，主要思想是从文法的开始符号出发，反复连续使用产生式，对非终结符进行替换和展开。
- 通常用大写字母表示非终结符，小写字母表示终结符， α 、 β 、 γ 等代表由终结符和非终结符号的并集的闭包中的元素组成的符号串。
 - 例如： $E \rightarrow i|EAE$ $A \rightarrow +|*$ 是上下文无关语法， E 、 A 是非终结符， E 是开始符，而 i ， $+$ 和 $*$ 是终结符。





上下文无关文法

- 例如: $N = \{E\}$ $T = \{+, *, (,), -, id\}$ $S = E$
- 算术表达式的定义可以写为:
 - $E \rightarrow id$
 - $E \rightarrow E + E$
 - $E \rightarrow E * E$
 - $E \rightarrow (E)$
- E 代表算术表达式, i 代表变量。这四个产生式的后三个是递归的。
- 我们可以定义如下文法 G
 - $E \rightarrow E + E | E * E | (E) | i$
 - 开始符号为 E , 从 E 出发 $E \Rightarrow (E) \Rightarrow (E + E) \Rightarrow (E * E + E) \Rightarrow (i * E + E) \Rightarrow (i * i + E) \Rightarrow (i * i + i)$
 - 符号" $\Rightarrow$ "表示仅推导一步





上下文无关文法

- 例如，文法G1：
 - $S \rightarrow bA$
 - $A \rightarrow aA|a$
- 从符号S出发我们可以推出，
 - $S \Rightarrow bA \Rightarrow ba$
 - $S \Rightarrow bA \Rightarrow baA \Rightarrow baa$
 - ...
 - $S \Rightarrow bA \Rightarrow baA \Rightarrow \dots \Rightarrow ba \dots a$
- 归纳得出所有以b开头后面跟一个或者多个a的字符串 $L(G1) = \{ban | n \geq 1\}$





- 算术表达式的定义可以写为：

- $E \rightarrow id$
- $E \rightarrow E + E$
- $E \rightarrow E * E$
- $E \rightarrow (E)$

- $1+3*5$ 的生成过程

- $E \rightarrow E + E \rightarrow E + E * E \rightarrow 1 + 3 * 5$





上下文无关文法表示自然语言

例句: He gave me a book.

英语中的基本语法规则:

<句子> $\rightarrow$ <主语> <谓语> <间接宾语> <直接宾语>

<主语> $\rightarrow$ <代词> | <名词>

<谓语> $\rightarrow$ <动词>

<间接宾语> $\rightarrow$ <代词>

<直接宾语> $\rightarrow$ <冠词> <名词>

<代词> $\rightarrow$ He | me

<名词> $\rightarrow$ book

<动词> $\rightarrow$ gave

<冠词> $\rightarrow$ a | an | the



例句: He gave me a book.

根据上述语法规则对例句进行分析, 可推出该例句。

<句子> => <主语> <谓语> <间接宾语> <直接宾语>
=> <代词> <谓语> <间接宾语> <直接宾语>
=> He <谓语> <间接宾语> <直接宾语>
=> He <动词> <间接宾语> <直接宾语>
=> He gave <间接宾语> <直接宾语>
=> He gave <代词> <直接宾语>
=> He gave me <直接宾语>
=> He gave me <冠词> <名词>
=> He gave me a <名词>
=> He gave me a book





上下文无关语言

- 句子无需考虑上下文，就可以判断正确性（符合<主语><谓语><间接宾语><直接宾语>的规则）。
- Book gave me a book是合法的。

例句：He gave me a book.

英语中的基本语法规则：

<句子> → <主语> <谓语> <间接宾语> <直接宾语>

<主语> → <代词>|<名词>

<谓语> → <动词>

<间接宾语> → <代词>

<直接宾语> → <冠词> <名词>

<代词> → He|me

<名词> → book

<动词> → gave

<冠词> → a|an|the

$$A = \pi r^2$$

推导方法

- 最左推导：推导过程中始终最先代换最左侧的文法
- 最右推导：推导过程中始终最先代换最右侧的文法

$$\begin{aligned} & \diamond E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id} \\ & E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \end{aligned}$$

❖ 最左推导

$$\begin{aligned} E & \Rightarrow_{lm} -E \Rightarrow_{lm} -(E) \Rightarrow_{lm} -(E + E) \\ & \Rightarrow_{lm} -(\text{id} + E) \Rightarrow_{lm} -(\text{id} + \text{id}) \end{aligned}$$

❖ 最右推导（规范推导）

$$\begin{aligned} E & \Rightarrow_{rm} -E \Rightarrow_{rm} -(E) \Rightarrow_{rm} -(E + E) \\ & \Rightarrow_{rm} -(E + \text{id}) \Rightarrow_{rm} -(\text{id} + \text{id}) \end{aligned}$$





练习

- 设文法G的定义如下：
- 非终结符集合：{S, NP, VP, D, V, N}，终结符集合：{saw, the, dog, bone, a}
- 开始符：S
- 产生式： $S \rightarrow NP VP$, $NP \rightarrow D N \mid N$, $VP \rightarrow V N \mid V NP$, $V \rightarrow \text{saw}$, $D \rightarrow \text{the} \mid \text{a}$, $N \rightarrow \text{dog} \mid \text{bone}$
- 画出 “the dog saw a bone”这句话的句法树。





如何将形式语言与自然语言融合以增强大型语言模型的可控性

- 自然语言的表达性优势？
- 形式语言的精确性优势？





- 1 自然语言的表达性优势
- 自然语言是人类沟通的基础，它具有高度的表达性和灵活性。在LLM中，自然语言的这些特性使得模型能够理解和生成丰富的文本内容，从而在各种任务中展现出创造性。然而，自然语言的模糊性和歧义性也可能导致LLM在生成计划时产生不精确或不明确的指令。





- 2 形式语言的精确性优势
- 形式语言，如上下文无关文法（CFG）和下推自动机（PDA），提供了一种精确和结构化的方式来描述规则和过程。这种精确性使得形式语言在需要严格遵循特定规则的任务中非常有用，例如编程语言和数学逻辑。形式语言的这种精确性可以帮助确保生成的计划是有效且可执行的。





- 3 Formal-LLM框架实现两者的互补
- Formal-LLM框架通过将自然语言的表达性与形式语言的精确性相结合，旨在克服单一语言类型的局限性。用户首先使用自然语言描述任务要求，然后这些要求被转化为形式语言（CFG），最后通过PDA来确保LLM在生成计划时遵循这些精确的规则。这样，Formal-LLM不仅能够理解复杂的任务描述，还能够生成符合预期的、可执行的计划。
- Formal-LLM框架：通过整合自然语言的表达性和形式语言的精确性来增强LLM的可控性。
- Formal-LLM框架允许用户以自动机的形式定义任务的约束和要求，然后LLM在这些约束的指导下生成计划。





CFG（上下文无关文法）与 LLM（大型语言模型）的结合

- parserLLM 项目：基于 ReLLM 扩展，将领域扩展到 CFG。用户定义 CFG 后，利用 Lark 解析器生成器处理这些规则，将生成的部分输出送入流程，由解析器反馈所有可能的下个 token，再将 tokens 转换成等效的正则表达式对接 ReLLM，筛选出模型能够生成且语法正确的字符或短语，循环往复直至达到最大生成长度或遇到停止条件，适合生成编程语言和配置文件等特定语言结构的文本。





CFG（上下文无关文法）与 LLM（大型语言模型）的结合

- XGrammar框架
- 陈天奇团队推出的开源软件库，专为 LLM 设计，提供高效、灵活且可移植的结构化数据生成能力。
- 它基于 CFG 定义结构，支持递归组合以表示复杂结构，适合生成 JSON、SQL 等格式数据。XGrammar 通过字节级下推自动机优化解释 CFG，减少每 token 延迟，实现百倍加速，几乎无额外开销。





CFG（上下文无关文法）与 LLM（大型语言模型）的结合

- SynCode框架
- 专为 LLM 设计，通过语法增强生成技术来指导模型输出遵循特定语法结构的代码。
- 该框架特别适用于需要严格语法规则的应用场景，如 JSON、Python、Go 等编程语言中的数据序列化或代码生成。
- SynCode 通过与 CFG 的结合，解决了 LLM 生成结果中常见的语法错误和不可预测性问题，保证了生成代码的正确性和可靠性。





- 在自然语言处理的实际应用中，RAG（检索增强生成）可能会利用知识库中的知识来生成回答，而 CFG 可以用于对问题和知识库中的文本进行句法分析，帮助更好地理解文本的结构和语义，从而更准确地进行知识检索和回答生成。
- 例如，在一个智能问答系统中，可能会使用 CFG 对用户输入的问题进行句法分析，以便更准确地从知识库中检索相关信息，然后通过 RAG 技术将检索到的信息与语言模型相结合，生成最终的回答。





PFCG

- 结合上下文无关文法（CFG）中最左派生规则(left-most derivations)和不同的 rules 概率，计算所有可能的树结构概率，取最大值对应的树作为该句子的句法分析结果。
- 对最左派生规则的每一步都添加概率，这样整棵句法分析树的概率就是所有这些“独立”的概率的乘积。



PCFG: 例子

$S \rightarrow NP VP$	[.80]	$Det \rightarrow that$	[.05]	the	[.80]	a	[.15]
$S \rightarrow Aux NP VP$	[.15]	$Noun \rightarrow book$					[.10]
$S \rightarrow VP$	[.05]	$Noun \rightarrow ights$					[.50]
$NP \rightarrow Det Nom$	[.20]	$Noun \rightarrow meal$					[.40]
$NP \rightarrow Proper-Noun$	[.35]	$Verb \rightarrow book$					[.30]
$NP \rightarrow Nom$	[.05]	$Verb \rightarrow include$					[.30]
$NP \rightarrow Pronoun$	[.40]	$Verb \rightarrow want$					[.40]
$Nom \rightarrow Noun$	[.75]	$Aux \rightarrow can$					[.40]
$Nom \rightarrow Noun Nom$	[.20]	$Aux \rightarrow does$					[.30]
$Nom \rightarrow Proper-Noun Nom$	[.05]	$Aux \rightarrow do$					[.30]
$VP \rightarrow Verb$	[.55]	$Proper-Noun \rightarrow TWA$					[.40]
$VP \rightarrow Verb NP$	[.40]	$Proper-Noun \rightarrow Denver$					[.40]
$VP \rightarrow Verb NP NP$	[.05]	$Pronoun \rightarrow you$	[.40]	I	[.60]		





Probabilistic Context-Free Grammar (PCFG)

- Formal definition (Wiki)

Similar to a CFG, a probabilistic context-free grammar G can be defined by a quintuple:

$$G = (M, T, R, S, P)$$

where

- M is the set of non-terminal symbols
- T is the set of terminal symbols
- R is the set of production rules
- S is the start symbol
- P is the set of probabilities on production rules

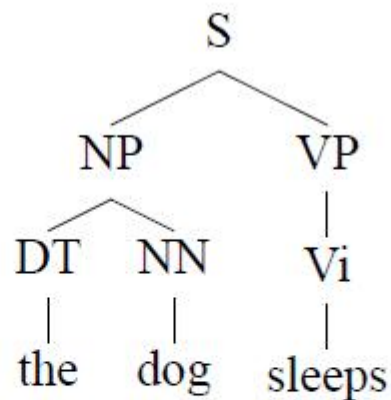


分析树的概率

Given a parse-tree $t \in T_G$ containing rules $\alpha_1 \rightarrow \beta_1, \alpha_2 \rightarrow \beta_2, \dots, \alpha_n \rightarrow \beta_n$, the probability of t under the PCFG is

$$p(t) = \prod_{i=1}^n q(\alpha_i \rightarrow \beta_i)$$

http://blog.csdn.net/qq_28031525



then we have

$$p(t) = q(S \rightarrow NP \ VP) \times q(NP \rightarrow DT \ NN) \times q(DT \rightarrow \text{the}) \times q(NN \rightarrow \text{dog}) \times q(VP \rightarrow Vi) \times q(Vi \rightarrow \text{sleeps})$$

http://blog.csdn.net/qq_28031525



乔姆斯基范式

- 在PCFG中添加对rules的限制(Chomsky Normal Form):

Definition 2 (Chomsky Normal Form) *A context-free grammar $G = (N, \Sigma, R, S)$ is in Chomsky form if each rule $\alpha \rightarrow \beta \in R$ takes one of the two following forms:*

- $X \rightarrow Y_1Y_2$ where $X \in N, Y_1 \in N, Y_2 \in N$.
- $X \rightarrow Y$ where $X \in N, Y \in \Sigma$.

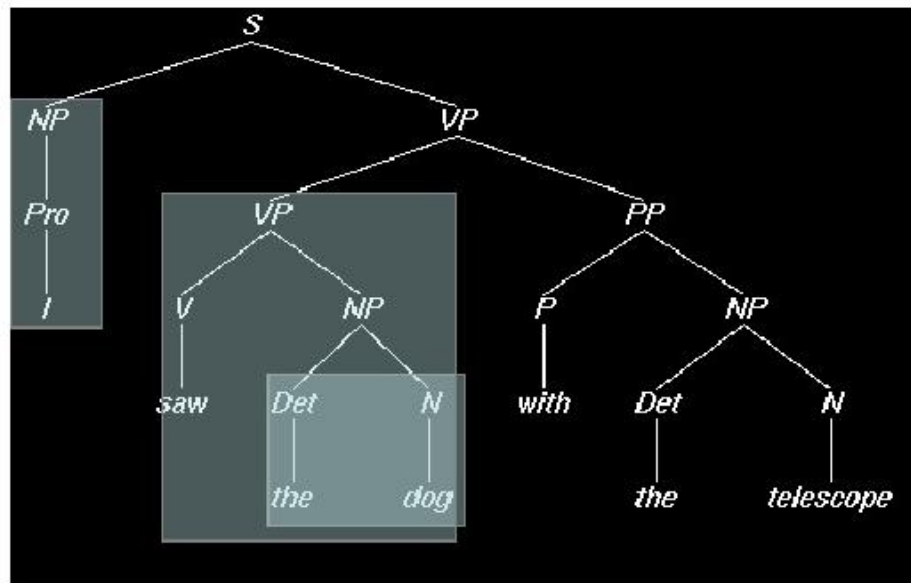
http://blog.csdn.net/qq_28031525



CFG: 句法分析树表示



(S (NP (Pro I))
(VP (VP (V saw)
(NP (Det the) (N dog)))
(PP (P with)
(NP (Det the) (N telescope))))))





PCFG的三个问题

- (1) 给定一个句子，估计产生句子的概率；
- (2) 在语句句法结构有歧义的情况下，如何快速选择最佳的句法分析；
- (3) 如何从语料库中训练文法的参数


$$\frac{A}{r} = \pi r^2$$

向内算法（自底向上）

- 输入: $G=(S,N,\Sigma,R,P)$, 字符串 $W = w_1w_2...w_n$
- 输出: $P(W|G) = \alpha_{1,n}(S)$
- 1、初始化: $\alpha_{i,i}(A) = P(A \rightarrow w_i), A \in N, 1 \leq i \leq n$
- 2、归纳计算: j从1到n, i从1到n-j, 重复下面计算

$$\alpha_{i,i+j}(A) = \sum_{B,C \in N} \sum_{i \leq k \leq i+j} P(A \rightarrow BC) \alpha_{i,k}(B) \alpha_{k+1,i+j}(C)$$

- 3、结束:
 $P(S \rightarrow w_1...w_n | G) = \alpha_{1,n}(S)$



寻找概率最大的句法分析树结构

- 问题是如何寻找概率最大的句法分析树结构?
- 1. 暴力搜索 (Brute-force method)
 - 暴力搜索Brute-force method(根据PCFG的rules, 暴力计算不同句法结构的概率, 选择概率最高的句法分析树作为结论), 缺陷在于随着句子的增长, 计算量指数增长:


$$\frac{A}{r} = \pi r^2$$



• 2. 动态规划 (dynamic programming)

• Viterbi算法

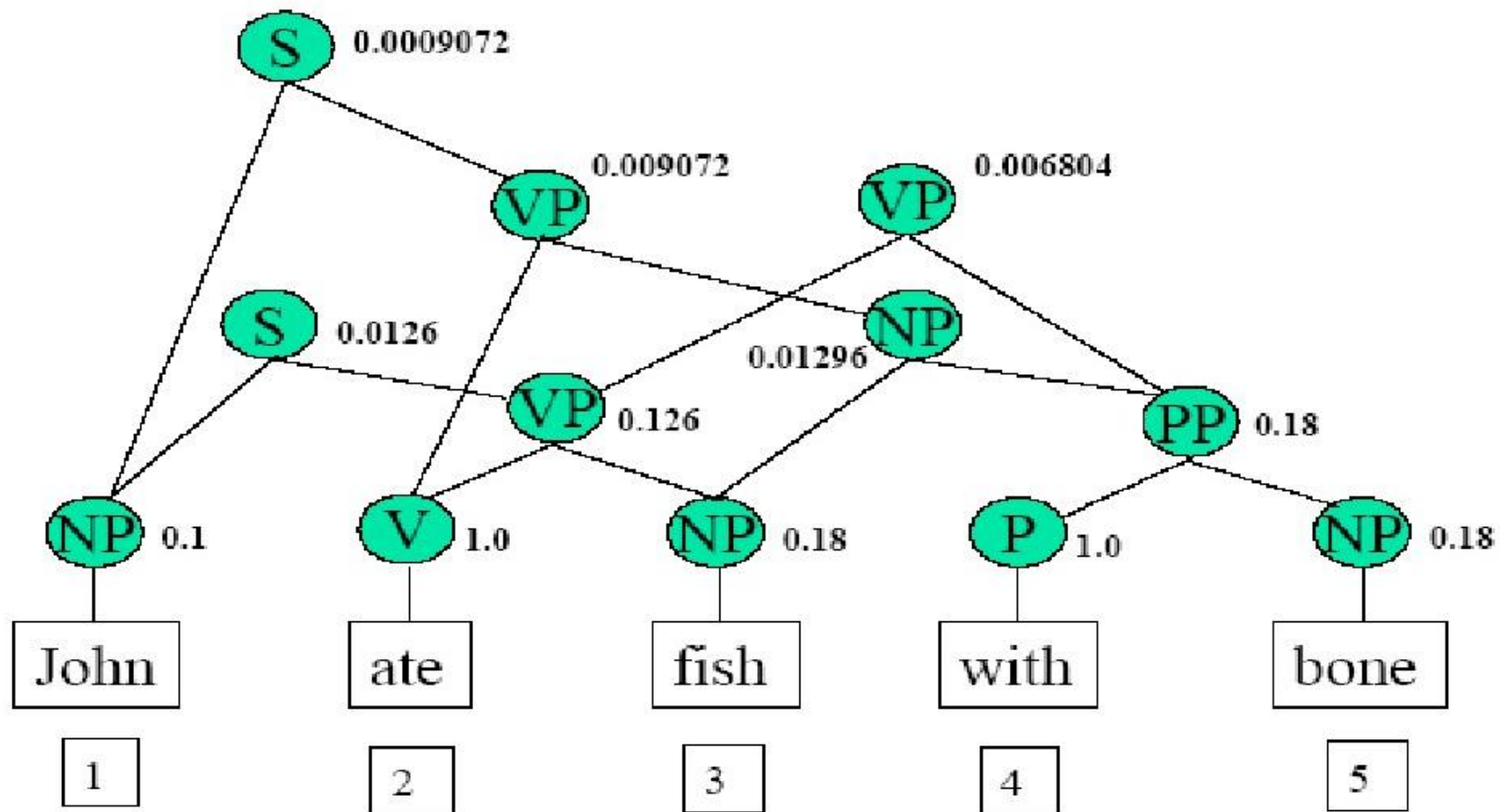
- 输入: $G=(S,N,\Sigma,R,P)$, 字符串 $W = w_1w_2\dots w_n$
- 输出: t^* (W 在 G 下最可能的分析树)
- 算法:
- 1、初始化 $\delta_{i,i}(A) = P(A \rightarrow w_i) \quad A \in N, 1 \leq i \leq n$
- 2、动态规划: j 从1到 n , i 从1到 $n-j$, 重复如下步骤

$$\delta_{i,i+j}(A) = \max_{B,C \in N; i \leq k \leq i+j} P(A \rightarrow BC) \delta_{i,k}(B) \delta_{k+1,i+j}(C)$$

$$\psi_{i,i+j}(A) = \arg \max_{B,C \in N; i \leq k \leq i+j} P(A \rightarrow BC) \delta_{i,k}(B) \delta_{k+1,i+j}(C)$$

- 3、结束 $P(t^*) = \delta_{1,n}(S)$
 t^* 的根节点为 S (文法开始符号); 从 $\psi_{1,n}(S)$ 开始回溯, 得到 S 的最优树结构
- $\psi_{i,i+j}(A)$ 记录了非终结符及其统摄的起止位置

Viterbi算法示例



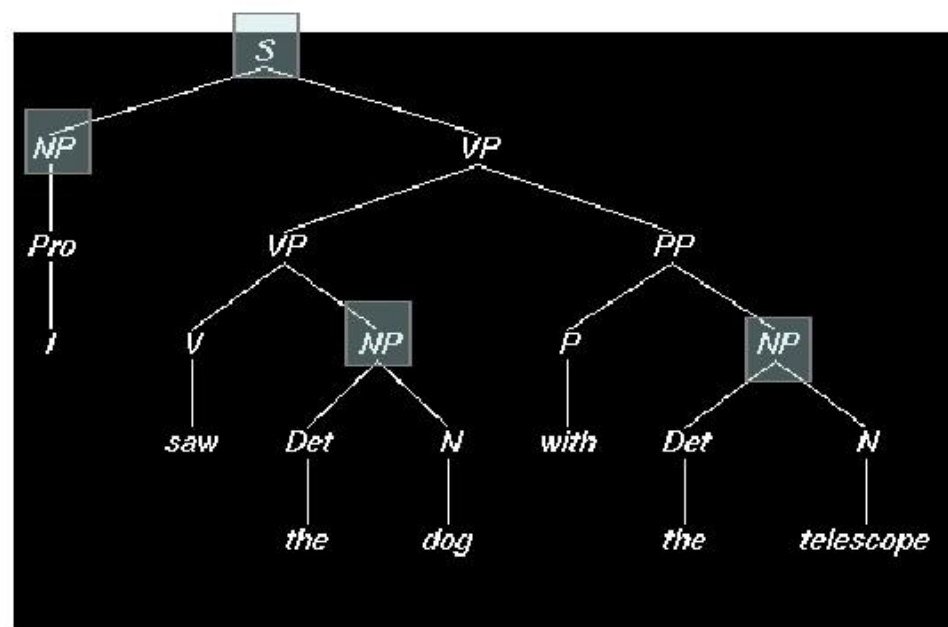
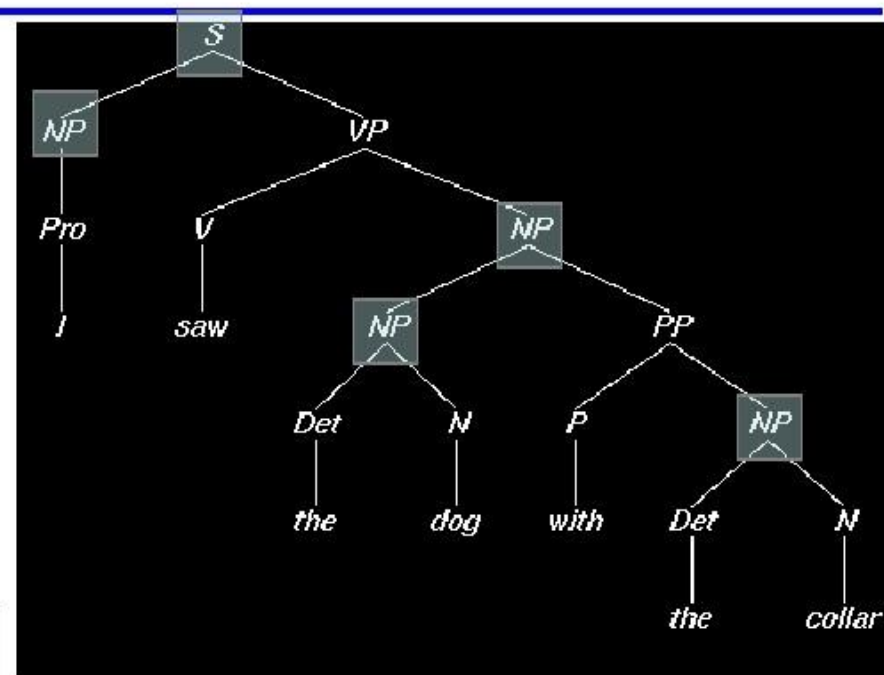
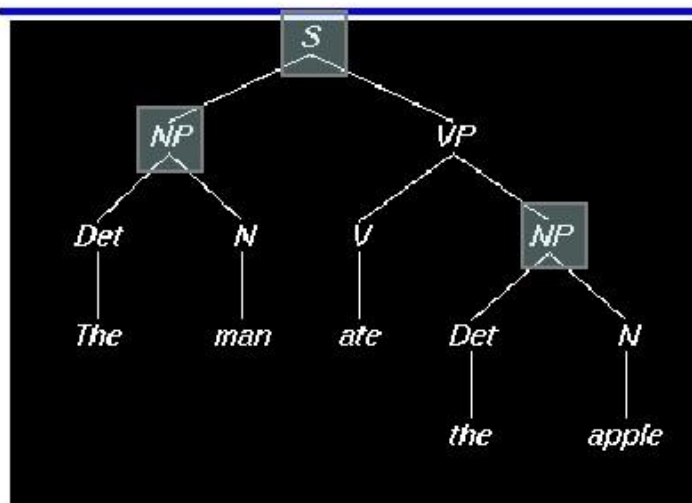


PFCG的参数训练

- 从树库直接统计——Treebank Grammar
 - 最大似然估计
 - 依赖于艰巨的工程：树库建设
- 向内向外算法
 - 迭代过程
 - 与初始参数相关



PCFG: 规则的概率估计



$S \rightarrow NP VP$	$3 / 3 = 1.0$
$NP \rightarrow Pro$	$2 / 9 = 0.22$
$NP \rightarrow Det N$	$6 / 9 = 0.67$
$NP \rightarrow NP PP$	$1 / 9 = 0.11$
$VP \rightarrow V NP$	$3 / 4 = 0.75$
$VP \rightarrow VP PP$	$1 / 4 = 0.25$
$PP \rightarrow P NP$	$2 / 2 = 1.0$





参数估计-EM算法

- EM (Expectation Maximum) 期望最大化
- 使用最大似然概率法来统计各条产生规则的概率，最后保证每条 $\sum\{N_{f \rightarrow N_j} N_g\} = 1.0$ 即可。

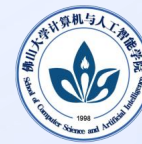




向内向外算法

- EM算法运用于PCFG的参数估计的具体算法。
 - 初始化：随机地给 $P(A \rightarrow \mu)$ 赋值，使得 $\sum_{\mu} P(A \rightarrow \mu) = 1$. 由此得到语法 G_0 . $i \leftarrow 0$.
 - EM步骤：
 - E步骤：计算期望值 $C(A \rightarrow BC)$ 和 $C(A \rightarrow a)$
 - M步骤：用E-步骤所得的期望值,利用：
$$\bar{P}(A \rightarrow \mu) = \frac{C(A \rightarrow \mu)}{\sum_{\mu} C(A \rightarrow \mu)}$$
重新估计 $P(A \rightarrow \mu)$,得到语法 G_{i+1}
 - 循环计算: $i++$,重复EM步骤,直至 $P(A \rightarrow \mu)$ 收敛.





课后自学：

- 自学Stanford Parser的句法分析功能。
- 对爬取的语料进行句法分析，掌握句法分析过程，观察句法分析结果。
- 自学Formal-LLM框架、XGrammar框架等



THANK YOU

